INPT
المعهد الوطني للبريد والمواصلات
ⵓⵙⴷⵍⵓ ⵓⵏⵎⵙⵙⵓ ⵏ ⵜⵙⵓⵏⵓⵔⵜ ⴰ ⵜⵎⵙⵙⵓⵔⵜ
Institut National des Postes et Télécommunications

MASTER DATA & IA
LE PREMIER MASTER DATA & IA EN ALTERNANCE

Course: Mathematics for AI

Gauss Method with Partial and Total Pivoting

Explanation, examples and Python program

Prepared by:

Otmane BELKHAIR

Academic year 2026–2027

October 31, 2026

1 Introduction

Solving a linear system

$$A\,x = b, \qquad A \in \mathbb{R}^{n\times n},\ x, b \in \mathbb{R}^n,$$

is one of the most common operations in science and artificial intelligence. Many algorithms, such as linear regression and optimisation methods, need to solve it.

The *Gauss method* changes the system into a simpler one, $Ux = c$, where U is upper triangular (zeros below the diagonal). Then we solve it from the bottom to the top (back substitution). The method needs every pivot (the number we divide by) to be different from zero. On a computer, a pivot that is very small also causes big rounding errors. *Pivoting* solves this problem by choosing a good pivot at each step.

This report presents:

- Gaussian elimination with **partial pivoting**, with a worked example;
- Gaussian elimination with **total (complete) pivoting**, with a worked example;
- a **Python program** for both methods.

2 Reminder: the Gauss method

Let $A^{(1)} = A$ and $b^{(1)} = b$. At step k ($k = 1, \dots, n-1$), if the pivot $a_{kk}^{(k)}$ is not zero, we compute the multipliers

$$m_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}, \qquad i = k+1, \dots, n,$$

and update

$$L_i \leftarrow L_i - m_{ik}\,L_k, \qquad a_{ij}^{(k+1)} = a_{ij}^{(k)} - m_{ik}a_{kj}^{(k)}, \qquad b_i^{(k+1)} = b_i^{(k)} - m_{ik}b_k^{(k)}.$$

After $n-1$ steps we get an upper triangular system $Ux = c$. We solve it from the last equation to the first one:

$$x_n = \frac{c_n}{u_{nn}}, \qquad x_i = \frac{1}{u_{ii}}\Big(c_i - \sum_{j=i+1}^{n} u_{ij}x_j\Big), \quad i = n-1, \dots, 1.$$

The number of operations is about $\frac{2}{3}n^3$.

Why pivoting? Take $\varepsilon x_1 + x_2 = 1,\ x_1 + x_2 = 2$ with $\varepsilon = 10^{-20}$. Without pivoting, $m_{21} = 10^{20}$ is huge, and the computer loses the information of the second row. If we swap the rows first, there is no problem. The two methods below use this idea.

3 Gauss method with partial pivoting

3.1 Principle

At step k, we look for the pivot only in **column** k, in the rows $k, \dots, n$. We take the entry with the largest absolute value:

$$\left|a_{pk}^{(k)}\right| = \max_{k \le i \le n} \left|a_{ik}^{(k)}\right|.$$

If $p \neq k$, we swap rows L_k and L_p (in both A and b), and then we eliminate as usual. Since the pivot is the largest entry of its column, all multipliers satisfy $|m_{ik}| \le 1$. So the numbers do not grow too much.

3.2 Algorithm

1. For $k = 1, \dots, n-1$:
   (a) find p such that $|a_{pk}| = \max_{i \ge k} |a_{ik}|$; if it is 0, the matrix has no unique solution;
   (b) swap $L_k \leftrightarrow L_p$;
   (c) for $i = k+1, \dots, n$: compute $m_{ik} = a_{ik}/a_{kk}$ and set $L_i \leftarrow L_i - m_{ik} L_k$.
2. Solve $Ux = c$ from the last equation to the first one.

In matrix form, this gives $PA = LU$, where P is a matrix that records the row swaps.

3.3 Worked example

Solve

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 14 \\ 3x_1 + 4x_2 + x_3 = 14 \\ 2x_1 + x_2 + 2x_3 = 10 \end{cases} \qquad \left(\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 3 & 4 & 1 & 14 \\ 2 & 1 & 2 & 10 \end{array}\right).$$

Step 1 – column 1. The largest entry in column 1 is 3 (row 2). We swap $L_1 \leftrightarrow L_2$:

$$\left(\begin{array}{ccc|c} 3 & 4 & 1 & 14 \\ 1 & 2 & 3 & 14 \\ 2 & 1 & 2 & 10 \end{array}\right).$$

Multipliers: $m_{21} = \frac{1}{3}$, $m_{31} = \frac{2}{3}$.

$$L_2 \leftarrow L_2 - \tfrac{1}{3} L_1, \quad L_3 \leftarrow L_3 - \tfrac{2}{3} L_1 \implies \left(\begin{array}{ccc|c} 3 & 4 & 1 & 14 \\ 0 & \frac{2}{3} & \frac{8}{3} & \frac{28}{3} \\ 0 & -\frac{5}{3} & \frac{4}{3} & \frac{2}{3} \end{array}\right).$$

Step 2 – column 2. Among $\left|\frac{2}{3}\right|$ and $\left|-\frac{5}{3}\right|$, the largest is $\frac{5}{3}$ (row 3). We swap $L_2 \leftrightarrow L_3$:

$$\left(\begin{array}{ccc|c} 3 & 4 & 1 & 14 \\ 0 & -\frac{5}{3} & \frac{4}{3} & \frac{2}{3} \\ 0 & \frac{2}{3} & \frac{8}{3} & \frac{28}{3} \end{array}\right).$$

Multiplier: $m_{32} = \dfrac{2/3}{-5/3} = -\frac{2}{5}$, and $L_3 \leftarrow L_3 + \frac{2}{5}L_2$:

$$\left(\begin{array}{ccc|c} 3 & 4 & 1 & 14 \\ 0 & -\frac{5}{3} & \frac{4}{3} & \frac{2}{3} \\ 0 & 0 & \frac{16}{5} & \frac{48}{5} \end{array}\right).$$

Solving from the bottom.

$$x_3 = \frac{48/5}{16/5} = 3, \qquad -\tfrac{5}{3}x_2 + \tfrac{4}{3}\cdot 3 = \tfrac{2}{3} \Rightarrow x_2 = 2, \qquad 3x_1 + 4\cdot 2 + 3 = 14 \Rightarrow x_1 = 1.$$

$$\boxed{x = (1,\ 2,\ 3)^T}$$

4 Gauss method with total pivoting

4.1 Principle

At step k, we look for the pivot in the **whole remaining square** $(a_{ij}^{(k)})_{i,j\geq k}$:

$$\left|a_{pq}^{(k)}\right| = \max_{i,j\geq k}\left|a_{ij}^{(k)}\right|.$$

Then we swap rows $k \leftrightarrow p$ *and* columns $k \leftrightarrow q$. When we swap two columns, the **unknowns change places**. We must remember this order and put the unknowns back in place at the end. In matrix form: $PAQ = LU$, with $x = Qy$.

4.2 Algorithm

1. Start with the list $\sigma = (1, 2, \ldots, n)$ (the order of the unknowns).
2. For $k = 1, \ldots, n-1$:
   (a) find (p, q) with the largest $|a_{ij}|$ for $i, j \geq k$; if it is 0, the matrix has no unique solution;
   (b) swap rows $L_k \leftrightarrow L_p$ (in A and b);
   (c) swap columns $k \leftrightarrow q$ and swap $\sigma_k \leftrightarrow \sigma_q$;
   (d) eliminate below the pivot as before.
3. Solve $Uy = c$ from the last equation to the first one; then put $x_{\sigma_k} = y_k$.

4.3 Worked example

We use the same system, to compare the two methods:

$$\left(\begin{array}{ccc|c} 1 & 2 & 3 & 14 \\ 3 & 4 & 1 & 14 \\ 2 & 1 & 2 & 10 \end{array}\right), \qquad \text{unknowns order: } (x_1, x_2, x_3).$$

Step 1. The largest entry in the whole matrix is 4 at position $(2, 2)$. We swap $L_1 \leftrightarrow L_2$ and then columns $C_1 \leftrightarrow C_2$:

$$\left(\begin{array}{ccc|c} 3 & 4 & 1 & 14 \\ 1 & 2 & 3 & 14 \\ 2 & 1 & 2 & 10 \end{array}\right) \xrightarrow{C_1 \leftrightarrow C_2} \left(\begin{array}{ccc|c} 4 & 3 & 1 & 14 \\ 2 & 1 & 3 & 14 \\ 1 & 2 & 2 & 10 \end{array}\right), \qquad \text{unknowns: } (x_2, x_1, x_3).$$

Multipliers: $m_{21} = \frac{1}{2}$, $m_{31} = \frac{1}{4}$.

$$\left(\begin{array}{ccc|c} 4 & 3 & 1 & 14 \\ 0 & -\frac{1}{2} & \frac{5}{2} & 7 \\ 0 & \frac{5}{4} & \frac{7}{4} & \frac{13}{2} \end{array}\right).$$

Step 2. The remaining sub-matrix is $\begin{pmatrix} -\frac{1}{2} & \frac{5}{2} \\ \frac{5}{4} & \frac{7}{4} \end{pmatrix}$; its largest entry in absolute value is $\frac{5}{2}$, located in the first row, second column. No row swap is needed, but we swap columns $C_2 \leftrightarrow C_3$:

$$\left(\begin{array}{ccc|c} 4 & 1 & 3 & 14 \\ 0 & \frac{5}{2} & -\frac{1}{2} & 7 \\ 0 & \frac{7}{4} & \frac{5}{4} & \frac{13}{2} \end{array}\right), \qquad \text{unknowns: } (x_2, x_3, x_1).$$

Multiplier: $m_{32} = \dfrac{7/4}{5/2} = \frac{7}{10}$, and $L_3 \leftarrow L_3 - \frac{7}{10} L_2$:

$$\left(\begin{array}{ccc|c} 4 & 1 & 3 & 14 \\ 0 & \frac{5}{2} & -\frac{1}{2} & 7 \\ 0 & 0 & \frac{8}{5} & \frac{8}{5} \end{array}\right).$$

Solving from the bottom (the unknowns are now in the order $y = (x_2, x_3, x_1)$):

$$x_1 = \frac{8/5}{8/5} = 1, \qquad \tfrac{5}{2} x_3 - \tfrac{1}{2} \cdot 1 = 7 \Rightarrow x_3 = 3, \qquad 4x_2 + 3 + 3 \cdot 1 = 14 \Rightarrow x_2 = 2.$$

We put the unknowns back in order and get

$$\boxed{x = (x_1, x_2, x_3)^T = (1,\ 2,\ 3)^T}$$

This is the same solution as with partial pivoting.

5 Comparison of the two methods

	Partial pivoting	**Total pivoting**
Where we look for the pivot	column k	the whole remaining square
Swaps	rows only	rows and columns
Unknowns change places	no	yes (we must remember it)
Matrix form	$PA = LU$	$PAQ = LU$
Cost of the search	small	bigger
Accuracy	good	the best
Use	most common choice	difficult or sensitive systems

Partial pivoting is almost always enough, and it is the usual choice in numerical libraries. Total pivoting is a little more accurate, but it costs more and is more complicated, so it is rarely needed.

6 Python program

The code below (file `gauss_pivoting.py`) uses NumPy. It has three functions: `gauss_partial` (partial pivoting), `gauss_total` (total pivoting) and `back_substitution` (solving from the last equation to the first one).

Listing 1: Gaussian elimination with partial and total pivoting

```python
import numpy as np


def back_substitution(U, c):
    """Solve U x = c where U is upper triangular."""
    n = len(c)
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (c[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x


def gauss_partial(A, b, tol=1e-12):
    """Gaussian elimination with partial (row) pivoting."""
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)
    for k in range(n - 1):
        # pivot = largest |a_ik| in column k, rows k..n-1
        p = k + np.argmax(np.abs(A[k:, k]))
        if abs(A[p, k]) < tol:
            raise ValueError("Matrix is singular")
        if p != k:                                  # row swap
            A[[k, p]] = A[[p, k]]
            b[[k, p]] = b[[p, k]]
        for i in range(k + 1, n):                   # elimination
            m = A[i, k] / A[k, k]
            A[i, k:] -= m * A[k, k:]
            b[i] -= m * b[k]
    if abs(A[n - 1, n - 1]) < tol:
        raise ValueError("Matrix is singular")
```

```
    return back_substitution(A, b)


def gauss_total(A, b, tol=1e-12):
    """Gaussian elimination with total (complete) pivoting."""
    A = np.array(A, dtype=float)
    b = np.array(b, dtype=float)
    n = len(b)
    perm = np.arange(n)        # perm[k] = original index of the unknown
        in column k
    for k in range(n - 1):
        # pivot = largest |a_ij| in the sub-matrix A[k:, k:]
        sub = np.abs(A[k:, k:])
        i, j = np.unravel_index(np.argmax(sub), sub.shape)
        i, j = i + k, j + k
        if abs(A[i, j]) < tol:
            raise ValueError("Matrix is singular")
        if i != k:                                  # row swap
            A[[k, i]] = A[[i, k]]
            b[[k, i]] = b[[i, k]]
        if j != k:                                  # column swap
            A[:, [k, j]] = A[:, [j, k]]
            perm[[k, j]] = perm[[j, k]]
        for r in range(k + 1, n):                   # elimination
            m = A[r, k] / A[k, k]
            A[r, k:] -= m * A[k, k:]
            b[r] -= m * b[k]
    if abs(A[n - 1, n - 1]) < tol:
        raise ValueError("Matrix is singular")
    y = back_substitution(A, b)
    x = np.zeros(n)
    x[perm] = y                # undo the column permutation
    return x


if __name__ == "__main__":
    A = [[1, 2, 3],
         [3, 4, 1],
         [2, 1, 2]]
    b = [14, 14, 10]
    print("Partial pivoting:", gauss_partial(A, b))
    print("Total pivoting  :", gauss_total(A, b))
    print("NumPy reference :", np.linalg.solve(A, b))

    # Stress test: tiny pivot
    eps = 1e-20
    A2 = [[eps, 1], [1, 1]]
    b2 = [1, 2]
    print("Tiny pivot, partial:", gauss_partial(A2, b2))
    print("Tiny pivot, total  :", gauss_total(A2, b2))
```

6.1 Results

We run the program on the example of Sections 3 and 4:

```
Partial pivoting: [1. 2. 3.]
Total pivoting  : [1. 2. 3.]
```

```
NumPy reference : [1. 2. 3.]
Tiny pivot, partial: [1. 1.]
Tiny pivot, total  : [1. 1.]
```

Both programs give the same result as our hand computation and as NumPy's `np.linalg.solve`. They also work well when the pivot is very small (10^{-20}), where the Gauss method without pivoting gives a wrong answer.

7 Conclusion

Pivoting is a simple and important improvement of the Gauss method. It avoids dividing by zero or by very small numbers, so the rounding errors stay small. Partial pivoting is cheap and accurate enough in most cases. Total pivoting is the most accurate, but it needs more work. We showed both methods with an example and wrote them in Python, and the results are the same.

References

[1] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.

[2] L. N. Trefethen, D. Bau, *Numerical Linear Algebra*, SIAM, 1997.

[3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.